Pruebas de Rendimiento del Software de Simulación Numérica IXCHEL2D

M.C. Edwin Enrique Pérez Rodríguez
Dr. Juan Carlos Cajas García
ENES-Mérida

8 de abril de 2026

Resumen

Se presentan las pruebas de escalamiento de datos de IXCHEL2D al utilizar la interfaz de programación de aplicaciones (API por sus siglas en inglés) de OpenACC. Se mide el tiempo de ejecución que le lleva a IXCHEL2D resolver el caso de un flujo dentro de una cavidad con paredes no deslizantes y se calcula la aceleración obtenida. Las mediciones se realizan utilizando unidades de procesamiento central multinúcleo (CPU multicore, por sus siglas en inglés) y unidades de procesamiento gráfico (GPU por sus siglas en inglés).

1. Introducción

IXCHEL2D es un software de simulación de termofluidos, discretizado mediante volúmenes de control y el esquema de ley de potencias. Utiliza mallas cartesianas escalonadas, resuelve las ecuaciones de Navier-Stokes y las de balance de energía para fluidos incompresibles mediante el uso del algoritmo SIMPLE. Está desarrollado en el lenguaje de programación de Fortran y la implementación del código paralelo se ha escrito utilizando la API de OpenACC.

En su etapa inicial, la parte de código paralelo de IXCHEL2D fue desarrollada con la API de OpenMP y sólo se podía ejecutar en procesadores multinúcleos. Con la alta madurez y estabilidad que han alcanzado las GPUs, se decidió desarrollar una implementación de IXCHEL2D que pudiera correr en estos dispositivos.

Un objetivo muy claro en el equipo de desarrollo es que IXCHEL2D se pueda ejecutar tanto en clusters de procesadores multinúcleos como en GPUs. La API de OpenACC permite desarrollar código paralelo sin la necesidad de escribir una versión para procesadores multinúcleo y otra para GPUs. El trabajo de implementar código para un dispositivo en particular se lleva a cabo por el compilador y la API de OpenACC. Debido a esta flexibilidad que ofrece OpenACC se decidió incorporarla como parte de las tecnologías que integran el núcleo de IXCHEL2D.

2. Implementación

La primera versión desarrollada de IXCHEL2D utilizaba la API de OpenMP. Una característica clave que comparte con OpenACC es que ambas permiten paralelizar código secuencial de manera muy sencilla. Otra característica importante a resaltar de ambas interfaces es la similitud de la sintaxis que definieron en su implementación. Esto permite que la migración del código escrito en OpenMP hacia OpenACC pueda realizarse sin mayores complicaciones.

En el siguiente bloque de código se puede observar un ejemplo de cómo paralelizar un bloque de código en Fortran con OpenMP:

```
!$omp parallel
    ... código ejecutado por cada thread
!$omp end parallel
```

y este es un ejemplo de cómo se reescribiría para que sea ejecutado por OpenACC:

```
!$acc parallel
    ... código ejecutado por cada thread
!$acc end parallel
```

Uno de los retos que surgen al desarrollar código para los procesadores gráficos es tener que escribir kernels encargados de ejecutar tareas específicas, luego se tiene que configurar como repartir el trabajo del kernel: cuántos bloques de threads se planean utilizar y cada bloque cuántos threads va a tener. Si no se tiene cuidado en la distribución de la carga de trabajo entre los bloques y threads puede originar que la aplicación se ejecute en más tiempo de lo esperado.

OpenACC resuelve el problema al facilitar la escritura de código para ejecutarse en una GPU al abstraer completamente el diseño e implementación de los kernels y la repartición eficiente de la carga de trabajo, esto quiere decir, que en tiempo de compilación OpenACC crea los kernels necesarios y distribuye la carga de trabajo de manera óptima.

Por ejemplo, la conocida operación SAXPY, se puede paralelizar en CUDA Fortran de la siguiente forma:

```
! Definición del kernel
attributes(global) subroutine ksaxpy( n, a, x, y )
    real, dimension(*) :: x,y
    real, value :: a
    integer, value :: n, i
    i = (blockidx%x-1) * blockdim%x + threadidx%x
    if( i <= n ) y(i) = a * x(i) + y(i)
end subroutine
```

La configuración necesaria para ejecutar el kernel y la manera en que se llama a este último se puede observar en el siguiente bloque de código:

```
program ksaxpy_cuda
    use cudafor
```

```
        implicit none

        ...

        ! Configurar ejecucion
        ! Numero de threads por bloque
        block = dim3(256, 1, 1)
        ! Numero total de bloques
        grid  = dim3((N + block - 1)/block, 1, 1)

        ! Ejecutar el kernel
        call ksaxpy<<<grid, block>>>( N, a, d_x, d_y )
        call cudaDeviceSynchronize()

        ...

    end program ksaxpy_cuda
```

El código mostrado anteriormente se puede refactorizar con OpenACC sin tanta complicación de la siguiente manera:

```
    program ksaxpy_cuda
        !$acc parallel loop
        do i = 1, N
            y(i) = a * x(i) + y(i)
        end do
        !$acc end parallel
    end program ksaxpy_cuda
```

Al delegar a OpenACC el diseño e implementación de los kernels así como de repartir la carga de trabajo permite al investigador o desarrollador concentrarse en la implementación matemático-computacional de la aplicación y dedicar poco tiempo a temas de diseño y arquitectura de software para unidades gráficas de procesamiento.

Otro gran reto en el desarrollo de código para GPUs es el movimiento de datos entre la CPU y la GPU. OpenACC cuenta con directivas que permiten mover datos de manera muy sencilla. Retomando el ejemplo de la rutina SAXPY, si se usa el paradigma de CUDA en FORTRAN, se necesita reservar memoria para los arreglos tanto en la CPU como la GPU. Luego hay que hacer el movimiento de datos de los arreglos entre ambos dispositivos de forma explícita. Un ejemplo de esto lo podemos observar en el siguiente bloque de código:

```
    program ksaxpy_cuda
        use cudafor

        implicit none

        ! Arreglos localizados en CPU
        real, allocatable :: x(:), y(:)
```

```
        ! Arreglos localizados en la GPU
        real, device, allocatable :: d_x(:), d_y(:)

        ! Se reserva memoria en la CPU
        allocate(x(N), y(N))
        ! Se reserva memoria en la GPU
        allocate(d_x(N), d_y(N))

        ! Copiar datos al GPU
        d_x = x
        d_y = y

        ...

        call ksaxpy<<<grid, block>>>( N, a, d_x, d_y )

        ...

        ! Copiar resultado a la CPU
        y = d_y

        ...

    end program ksaxpy_cuda
```

La implementación de OpenACC permite simplificar el trabajo realizado en el bloque de código anterior:

```
    program ksaxpy_cuda

        ...

        ! Solo es necesario almacenar arreglos en la CPU
        real, allocatable :: x(:), y(:)
        allocate(x(N), y(N))

        ! Movimiento de los datos entre CPU y GPU
        !$acc data copy(y(1:N)) copyin(x(1:N))
        !$acc parallel loop
        do i = 1, N
            y(i) = a * x(i) + y(i)
        end do
        !$acc end data
    end program ksaxpy_cuda
```

donde la directiva `copy` mueve los datos desde la CPU a la GPU y viceversa, y `copyin` solo mueve datos desde la CPU a la GPU.

Las grandes ventajas ofrecidas por la sintaxis de OpenACC ha permitido al equipo de

IXCHEL2D mantener el código de la aplicación compacto, legible y fácil de mantener. Esto lo podemos observar por ejemplo en como se ha organizado el movimiento de los múltiples arreglos que se requieren en los cálculos:

```
!$acc data copy(&
!$acc &    u(1:mi,1:nj+1),v(1:mi+1,1:nj),                  &
!$acc &    pres(1:mi+1,1:nj+1),temp(1:mi+1,1:nj+1),        &
!$acc &    corr_pres(1:mi+1,1:nj+1),                       &
!$acc &    u_ant(1:mi,1:nj+1),v_ant(1:mi+1,1:nj),          &
!$acc &    temp_ant(1:mi+1,1:nj+1),b_o(1:mi+1,1:nj+1)      &
!$acc &    )
```

Además OpenACC permite agregar configuraciones extras que apoyen a mejorar el rendimiento de la aplicación, por ejemplo, que paralelice un bloque de ciclos `for` anidados. En este caso, con la directiva `collapse` se solicita que los dos bucles anidados se colapsen en uno solo.

```
! Se ensambla la velocidad v en direcci\'on x
!
!$acc parallel loop gang collapse(2)
do jj = 2, nj-1
    do ii = 2, mi
    call ensambla_velv_x(deltaxv,deltayv,deltaxu,&
            &deltayp,fexp,feyp,feyv,gamma_momen,&
            &fuente_con_v,fuente_lin_v,&
            &v,v_ant,u,&
            &temp,pres,Riy,dt,rel_vel,&
            &AI,AC,AD,Rx,av,&
            &ii,jj&
            &)
    end do
end do
```

3. Resultados

Las pruebas de escalamiento se han realizado en una estación de trabajo y en un servidor virtual de alto rendimiento. Las características de hardware de la estación de trabajo son:

- Un procesador Intel(R) Core(TM) i7-9700 @ 3.00GHz de 8 núcleos
- 32 GB de memoria RAM DDR4
- Una tarjeta gráfica GeForce RTX 5070 equipada con 12 GB de VRAM GDDR7[1].

El servidor virtual cuenta con las siguientes características de hardware:

- Dos procesadores Intel(R) Xeon(R) Gold 6438M (4 núcleos asignados en la máquina virtual),

[1] La RTX5070 cuenta con 48 Streaming Multiprocesors.

- 16 GB de memoria RAM
- Una tarjeta gráfica NVIDIA A40 con memoria GDDR6 ECC en configuración vGPU A40-8Q, asignando 8 GB dedicados de los 48 GB de la tarjeta física[2].

Se realiza una prueba de escalamiento de datos para observar como se comporta la aceleración de IXCHEL2D al aumentar la entrada de datos cuando se ejecuta en dispositivos multinúcleo y en unidades de procesamiento gráfico. IXCHEL2D recibe como entrada, entre otros datos, dos valores m_i y n_j que forman una malla de tamaño $m_i \times n_j$. Se consideran mallas cuadradas y los números de puntos que se usan para las pruebas son: $64, 128, 256, 512$.

3.1. Descripción del caso de prueba

El caso de prueba que se utilizará para realizar las pruebas de escalamiento es conocido como LidCavity. Simula el flujo en una cavidad con paredes no deslizantes donde se impone una velocidad uniforme en la tapa. Se considera el caso en 2D por lo que se modela numéricamente una placa cuadrada de lado unitario.

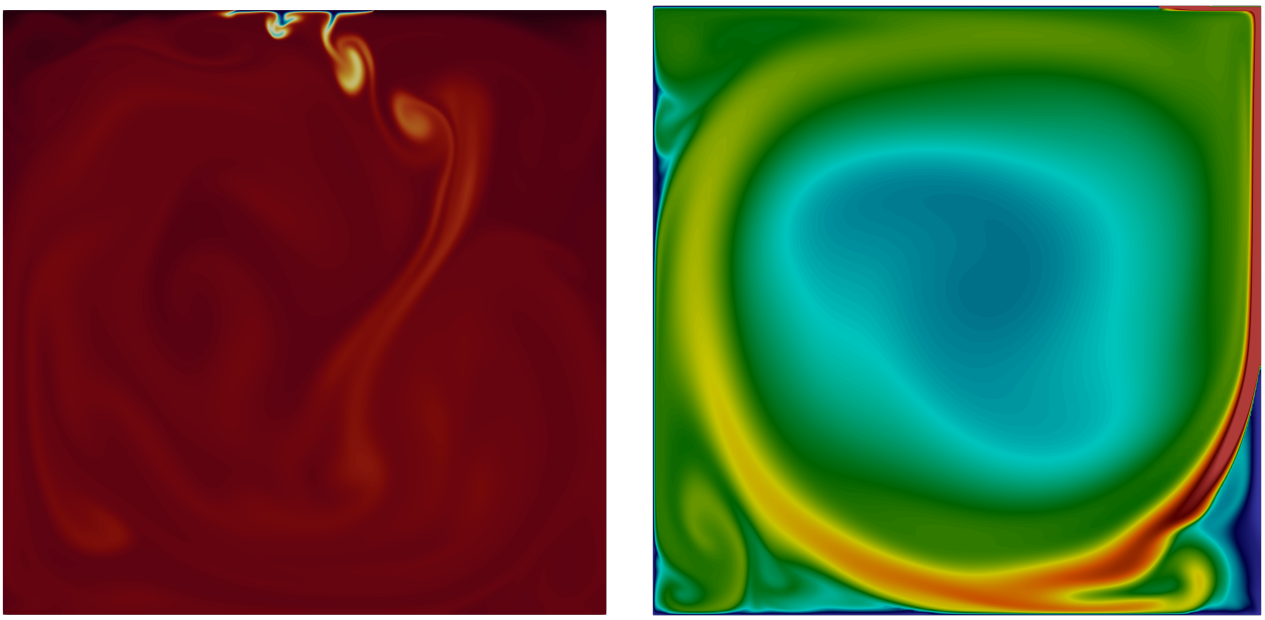

Figura 1: Simulaciones del flujo en una cavidad con paredes no deslizantes en 2D realizadas con IXCHEL2D.

[2]La A40-8Q es una VGPU time-sliced que proporciona acceso compartido a los 84 Streaming Multiprocessors de la GPU física gestionados por el vGPU manager del hypervisor. La memoria está particionada (8 GB exclusivos para la configuración 8Q), mientras que los recursos de cómputo se comparten dinámicamente entre las instancias vGPU activas. [1]

Número de Puntos	Core i7-9700	Xeon Gold 6438M
64×64	2.6	2.5
128×128	3.8	4.5
256×256	4.7	5.8
512×512	3.6	6.9

Tabla 1: Comparación de la aceleración obtenida con los procesadores Intel Core i7-9700 e Intel Xeon Gold 6438M con 8 cores cada uno.

3.2. Tiempo de Cómputo

Se presentan los resultados obtenidos. En los datos de la figura 2 se puede observar la comparación de la aceleración obtenida al escalar los datos usando el procesador Xeon Gold 6438M y el procesador Core i7-9700. La aceleración alcanzada por el procesador Xeon Gold 6438M no se ve afectada al incrementar el tamaño de datos, al menos hasta 512×512 puntos. En cambio, se puede notar que el procesador Core i7-9700 se mantiene escalando bien su aceleración hasta 256×256 puntos, pero esta decae al procesar 512×512 puntos. Esto se puede deber al lugar donde se almacenan los datos. Aunque el conjunto total de los datos se almacena en la memoria RAM, aquellos datos que se necesitan acceder de manera constante son movidos a la memoria caché.

La CPU cuenta con tres niveles de memoria caché: L1, L2 y L3, teniendo cada una una velocidad de acceso más rápida que la anterior y son más rápidas de acceder que la memoria RAM. La memoria caché L1 es la más rápida de las tres además es la más cercana a la CPU. La memoria caché L2 del procesador Core i7-9700 es 2 MiB y la L3 es de 12 MiB, en cambio la memoria caché L2 del procesador Xeon Gold 6438M es de 32 MiB y 32 MiB en la caché L3. La caída en el rendimiento del procesador Core i7-9700 se debe al costo a pagar por mover los datos a otro nivel de memoria, aumentando el tiempo de lectura/escritura.

De acuerdo a lo anterior, IXCHEL2D tiene un buen comportamiento tanto en equipos de escritorio como en clusters de alto rendimiento al utilizar multiprocesamiento, siendo la principal limitante las características del hardware de los equipos de cómputo no especializado. Ahora revisaremos los resultados del escalamiento de datos al comparar la aceleración alcanzada por el procesador Xeon Gold 6438M con las unidades de procesamiento gráfico A40-8Q y RTX-5070.

La figura 3 muestra que para un tamaño de 512×512 puntos la tarjeta RTX-5070 logra una aceleración de 12x y supera el speedup ideal del procesador Xeon Gold 6438M. Para este tamaño de puntos la carga masiva de cómputo compensa la latencia que se obtiene al mover grandes cantidades de datos desde la CPU hacia la GPU y viceversa. Sin embargo, con un tamaño de puntos más pequeños el costo de la transferencia de datos penaliza la aceleración y es mejor usar procesadores multinúcleo.

La tarjeta A40-8Q presenta un rendimiento por debajo de la RTX-5070. Esta GPU es la que más penalización presenta por la transferencia de datos, lo que provoca que para un tamaño de 64×64 puntos su rendimiento sea inferior al de la ejecución secuencial. Incluso para un tamaño de puntos grande como 512×512 apenas alcanza el rendimiento del procesador Xeon Gold 6438M.

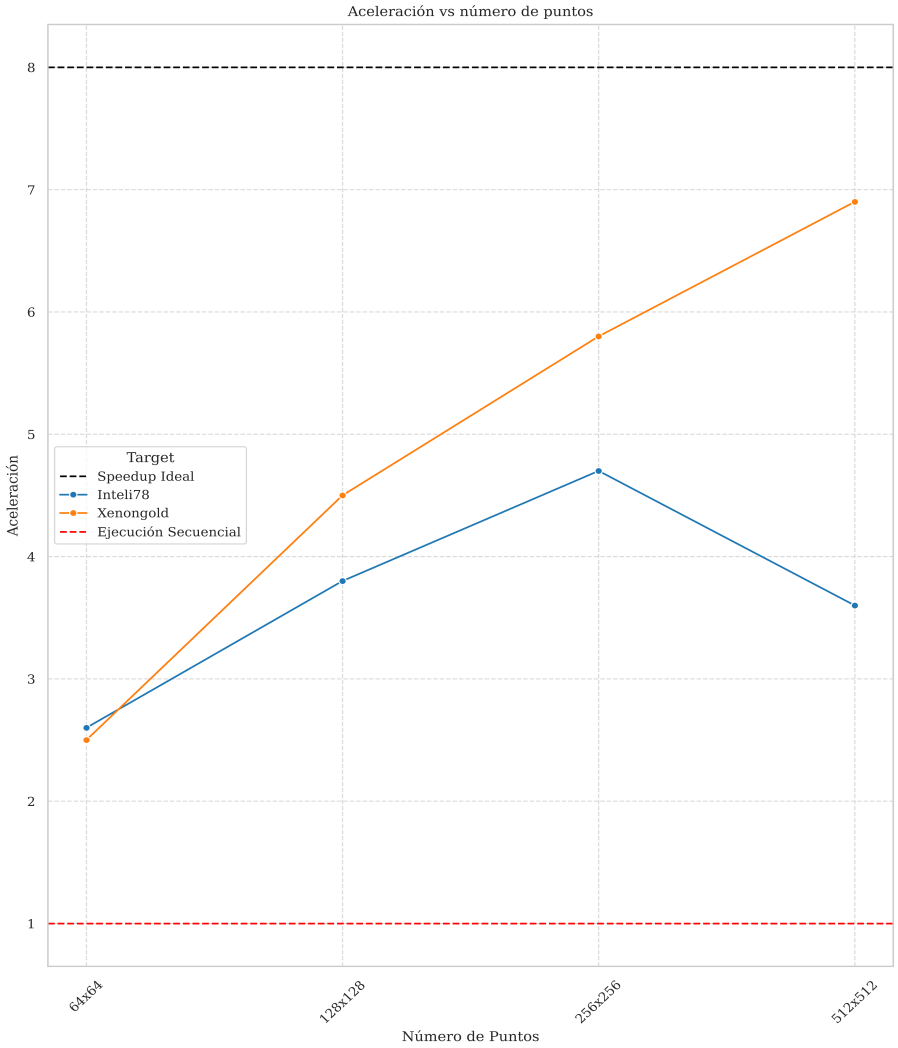

Figura 2: Comparación de la aceleración obtenida con los procesadores Intel Core i7-9700 e Intel Xeon Gold 6438M con 8 cores cada uno.

El bajo rendimiento de la tarjeta A40-8Q puede responder a dos factores: primero, al overhead inherente a la virtualización por time-slicing (compartición de los 84 SMs entre instancias vGPU activas); segundo, las políticas configuradas para el scheduling del hypervisor que determina la asignación temporal de recursos computacionales entre las instancias virtuales [2].

De acuerdo a los datos presentados en la figura 4, se obtienen resultados similares a la figura 3 pero esta vez utilizando el procesador Core i7-9700. La principal diferencia se encuentra en que debido a la limitación impuesta por el tamaño de la memoria caché del procesador Core i7-9700, para tamaños grandes del orden 512×512 puntos, ambas unidades de procesamiento gráfico superan el tiempo de ejecución de la unidad multinúcleo, aunque solo la RTX-5070 supera el speedup ideal del Core i7-9700.

Como se observa en la figura 2, IXCHEL2D tiene buen rendimiento de escalado en un cluster

Número de Puntos	Xeon Gold 6438M	A40-8Q	RTX-5070
64×64	2.5	0.68	1.3
128×128	4.5	1.8	3.0
256×256	5.8	3.4	5.2
512×512	6.9	6.8	12.0

Tabla 2: Comparación de la aceleración obtenida con el procesador Intel Xeon Gold 6438M con 8 cores, las unidades de procesamiento gráfica A40-8Q y RTX-5070.

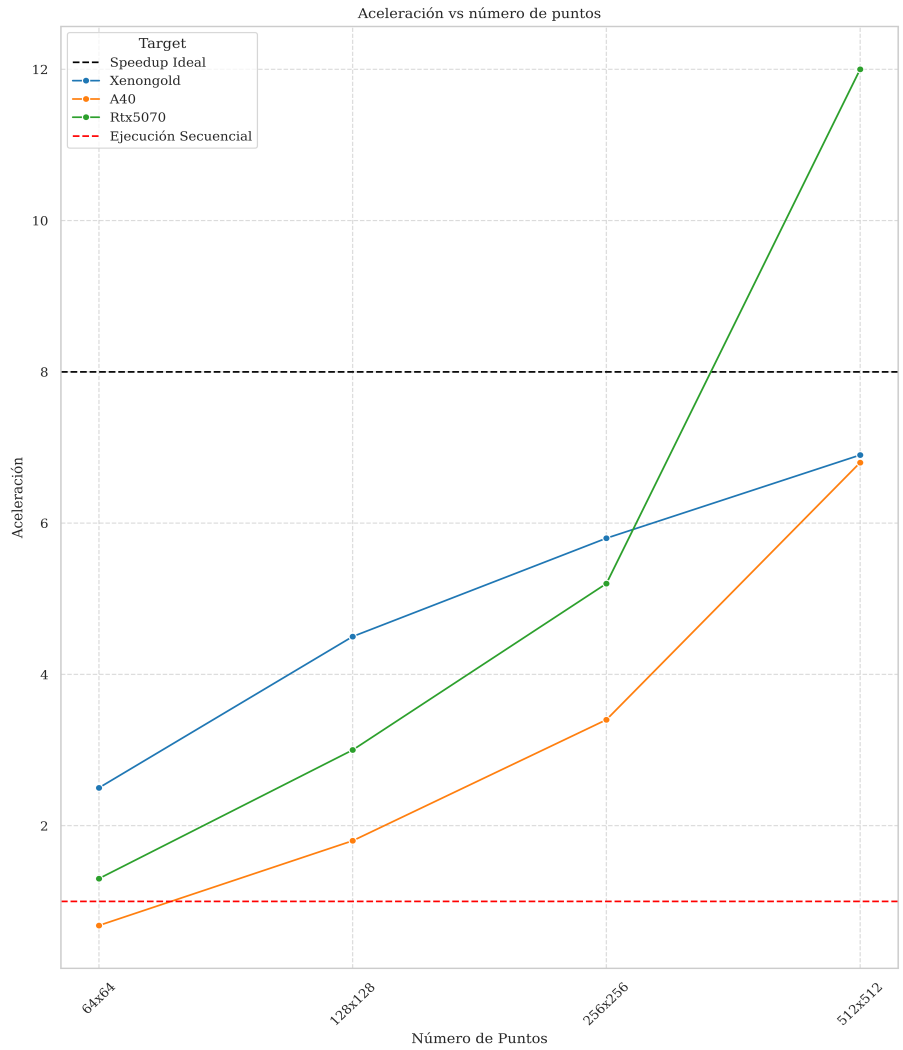

Figura 3: Comparación de la aceleración obtenida con el procesador Intel Xeon Gold 6438M con 8 cores, las unidades de procesamiento gráfica A40-8Q y RTX-5070.

de alto rendimiento que cuente con procesadores Xeon Gold 6438M y aunque no se cuente con una unidad de procesamiento gráfica, se consigue una buena aceleración con respecto al tiempo secuencial. Si no se tiene acceso a un cluster, aún es posible ejecutar IXCHEL2D en estaciones de trabajo personales como se vio con el procesador Core i7-9700, pero se tiene que tomar en cuenta que tendremos una reducción en el rendimiento.

Número de Puntos	Core i7-9700	A40-8Q	RTX-5070
64×64	2.6	0.58	1.1
128×128	3.8	1.6	2.6
256×256	4.7	3.1	4.8
512×512	3.6	6.0	11.0

Tabla 3: Comparación de la aceleración obtenida con el procesador Intel Core i7-9700 con 8 cores, las unidades de procesamiento gráfica A40-8Q y RTX-5070.

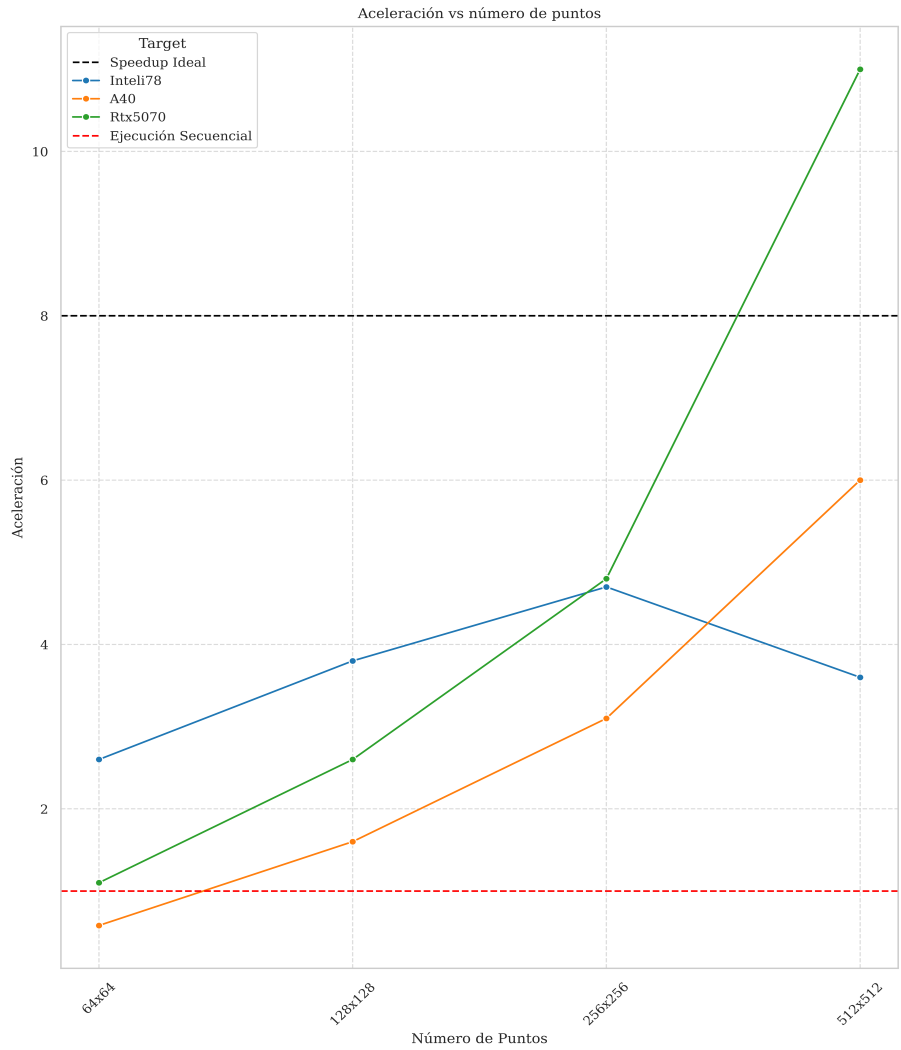

Figura 4: Aceleración OpenACC Multicore

Notamos tanto en la figura 3 como en la figura 4 que si contamos con una unidad de procesamiento gráfico, hay que tener presente si el tamaño de datos del problema es lo suficientemente grande para amortizar el costo de la transferencia de datos. Esto ayuda a decidir si conviene correr IXCHEL2D en uno u otro dispositivo. Aunque la tarjeta A40-8Q es mayoritariamente encontrada en clusters de supercómputo y centros de datos, su rendimiento está siendo superado por una tarjeta gráfica diseñada para videojuegos.

4. Agradecimientos

Se agradece el soporte técnico brindado por el Ing. Joaquín Morales Rosales, técnico académico de la ENES-Mérida UNAM y también por los recursos de cómputo de alto rendimiento provistos por la DGTIC-UNAM por medio del Ing. Pedro Bautista Fernández.

Referencias

[1] NVIDIA Corporation. *NVIDIA vGPU Software User Guide*. 2024. URL: https://docs.nvidia.com/vgpu/latest/grid-vgpu-user-guide/index.html#architecture-internal-grid-vgpu-time-sliced (visitado 27-02-2025).

[2] NVIDIA Corporation. *vGPU Features and Scheduling Policies*. 2024. URL: https://docs.nvidia.com/vgpu/knowledge-base/latest/vgpu-features.html#vgpu-schedulers (visitado 27-02-2025).